

Python Namespace

1. What is a Namespace

- A namespace is a system used by Python to store and manage names and their corresponding objects.
- A namespace is a place where Python keeps track of names like variables, functions, and classes and the objects they refer to.

2. Types of Namespace in Python

- 1) Built-in Namespace.
- 2) Global Namespace.
- 3) Local Namespace.
- 4) Enclosing Namespace.

3. Scope in Python

- Scope is the area of a program where a particular name (variable, function, etc.) can be accessed or used.
- Scope tells us where a variable can be used in the program.

○ Namespace vs Scope

Namespace	Scope
Where is the name stored/managed?	Where can I use the name
Example: Global Namespace, Local Namespace.	Example: Global Scope, Local Scope.

4. Global Namespace

- Global Namespace is the namespace that contains names defined at the main level of a Python program or module.
- When you create a variable, function, or class outside of any function or class, its is Global Namespace.
- These names can generally be accessed from different parts of the same module, subject to Python's scope rules.

```
name = "demo"    #global
age = 22         #global

def display():
    print(name)
    print(age)

display()
```

5. Local Namespace

- Local Namespace is the namespace that contains names created inside a function.
- When you create a variable, parameter, or other name inside a function, it belongs to that function's Local Namespace.
- These names are normally accessible only inside that function.

```
def display():
    name = "DemoUser"
    age = 22

    print(name)
    print(age)

display()
```

6. Built-in Namespace

- Built-in Namespace is the namespace that contains names provided by Python by default, such as built-in functions, classes, and exceptions.
- Python provides many functions and names that we can use without defining them ourselves.

```
name = "demo"  
  
print(name)  
print(len(name))
```

7. Enclosing Namespace

- Enclosing Namespace is the namespace of an outer function that contains a nested (inner) function.
- When a function is defined inside another function, the inner function can access names from the outer function.
- The names of the outer function are part of the Enclosing Namespace for the inner function.

```
def outer():  
    x = 10  
  
    def inner():  
        print(x)  
  
    inner()  
  
outer()
```

8. Shadowing

- Shadowing occurs when a name in a local or inner scope has the same name as a name in an outer scope.
- There are two variables with the same name x

```
x = 10

def test():
    x = 20
    print(x)

test()
print(x)
```

9. global Keyword

- The global keyword is used inside a function to tell Python that a variable belongs to the Global Namespace.
- If we want to modify the global variable, use global:

```
x = 10

def test():
    global x
    x = 20

test()
print(x)
```

10. LEGB Rule in Python

- LEGB is the order in which Python searches for a name (such as a variable or function) when it is used in a program.
- LEGB stands for – Local , Enclosing , Global , Built-in

Letter	Meaning	Where Python Searches
L	Local	Inside the current function
E	Enclosing	Inside the outer function
G	Global	At the main/module level
B	Built-in	Python's built-in names

- 1) L → Local
 - Python first checks the current function.

- 2) E → Enclosing
 - If Python doesn't find the name locally, it checks the outer function.

- 3) G → Global
 - If Python doesn't find the name in Local or Enclosing scope, it checks the Global Namespace.

- 4) B → Built-in
 - If Python doesn't find the name in Local, Enclosing, or Global, it searches the Built-in Namespace.